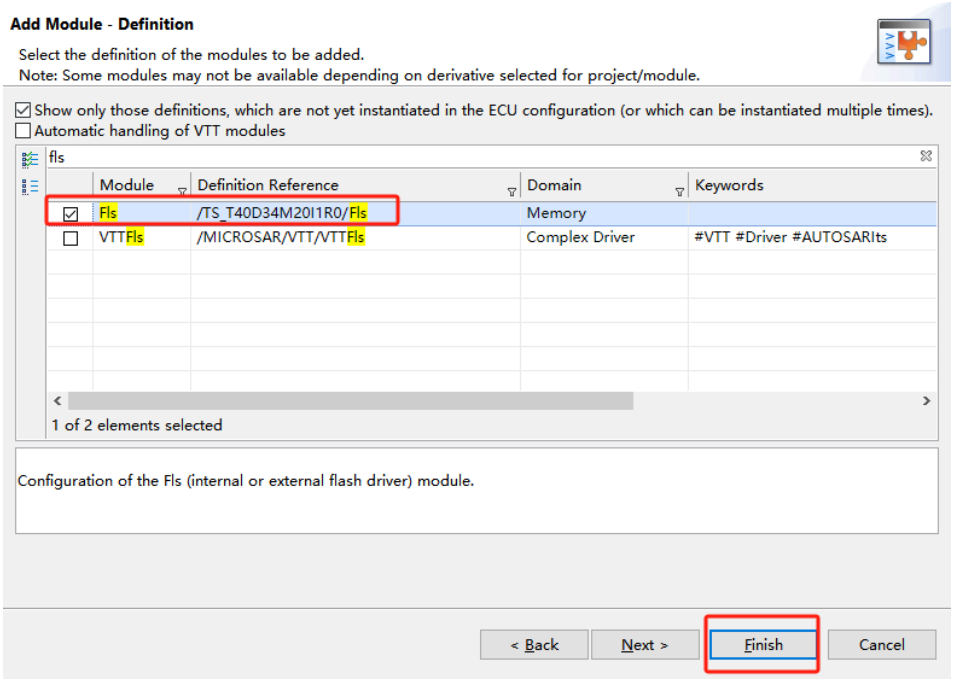
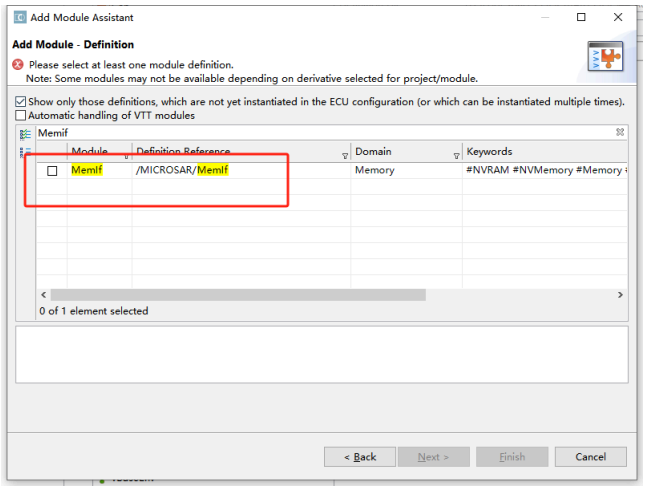
13、添加Fls

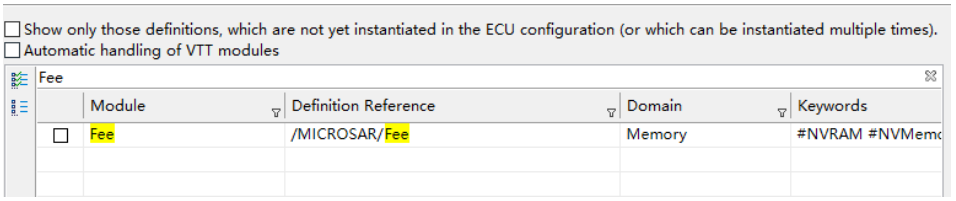
卖家'漩涡遨游aa' 专属资料，非在漩涡遨游aa 购买资料，不提供答疑(一次购买仅提供有限次数10次答疑)

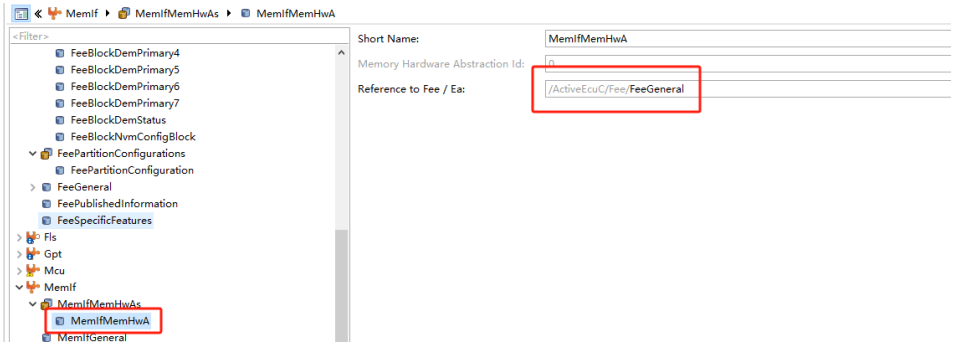
一、添加模块



注意：MemIf 和Fee都添加，MemIf应用Fee，但是Fee我们不消错，不生成，下一节才配置他

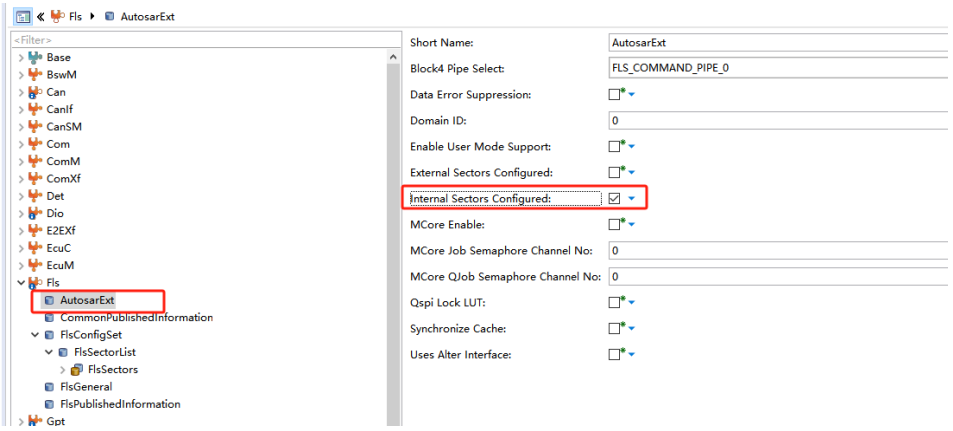






二、模块配置

勾选内部模块支持，因为我们用的是DFlash做测试，不用外部flash



2.1、DataFlash和CodeFlash概述

我们使用的K344 有

4096K CodeFlash, 起始地址0x00400000,

128K DataFlash, 起始地址0x10000000

	A	B	C	D	E	F	G	H	I	J	K
1						Implemented size [KB]					
2	Start address [Hex]	End address [Hex]	Max size [KB]	Description	Cache mode at reset	S32K312	S32K322	S32K342	S32K314	S32K324	S32K344
3	0x00000000	0x0000FFFF	64	ITCM_0	Non-cacheable	32	32	64	32	32	64
4	0x00000000	0x0000FFFF	64	ITCM_1	Non-cacheable		32			32	
5	0x00400000	0x00BFFFFF	8192	Program flash	Non-cacheable	2048	2048	2048	4096	4096	4096
6	0x10000000	0x1003FFFF	256	Data flash	Non-cacheable	128	128	128	128	128	128
7	0x11000000	0x1100FFFF	64	ITCM_0 Backdoor	Non-cacheable	32	32	64	32	32	64
8	0x11400000	0x1140FFFF	64	ITCM_1 Backdoor	Non-cacheable		32		32	32	
9	0x1B000000	0x1B001FFF	8	UTEST (OTP)	Non-cacheable	8	8	8	8	8	8
10	0x20000000	0x2001FFFF	128	DTCM_0	Non-cacheable	64	64	128	64	64	128
11	0x20000000	0x2001FFFF	128	DTCM_1	Non-cacheable		64			64	
12	0x20400000	0x20427FFF	160	SRAM0_less or equal 4MB	Non-cacheable	96	64	64	160	160	160
13	0x20428000	0x2044FFFF	160	SRAM1_less or equal 4MB	Non-cacheable				160	160	160
14	0x21000000	0x2101FFFF	128	DTCM_0 Backdoor	Non-cacheable	64	64	128	64	64	128
15	0x21400000	0x2141FFFF	128	DTCM_1 Backdoor	Non-cacheable		64		64	64	
16	0x40000000	0x401FFFFF	2048	AIPS_0	Non-cacheable	2048	2048	2048	2048	2048	2048
17	0x40200000	0x403FFFFF	2048	AIPS_1	Non-cacheable	2048	2048	2048	2048	2048	2048
18	0x40400000	0x405FFFFF	2048	AIPS_2	Non-cacheable		2048	2048	2048	2048	2048
19	0x67000000	0x670003FF	1	QuadSPI RX buffer	Non-cacheable		1	1	1	1	1
20	0x68000000	0x6FFFFFFF	131072	QuadSPI AHB (virtual code & data)	Non-cacheable		131072	131072	131072	131072	131072
21	0xE0000000	0xE00FFFFF	1024	Private Peripheral Bus	Non-cacheable	1024	1024	1024	1024	1024	1024
22	Total RAM [KB]					192	256	256	512	512	512
23											

2.2、配置第一块DataFlash

Physical Sector: dataflash共分为S000-S015， 16块，每块8K

Number of Sectors： Physical Sector开始，这个sector配置连续多个作为整体

Sector Hw Address: 块实际起始地址-0x10000000

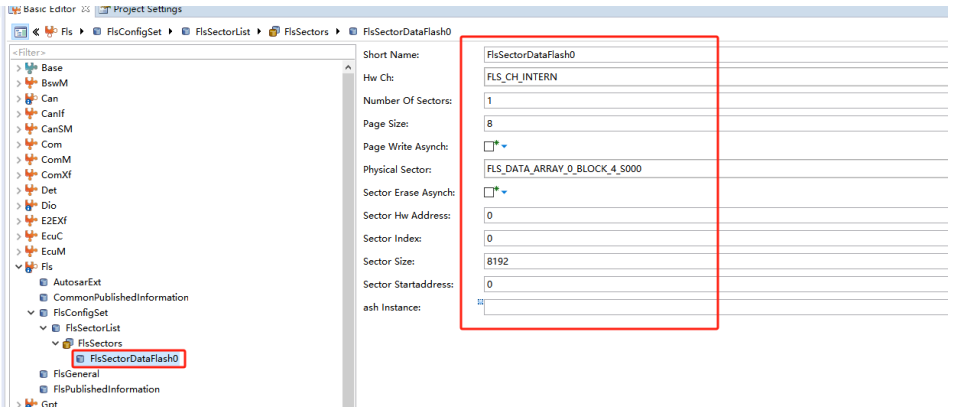
Sector index: 从0开始

Sector size: 8192

Sector Startaddress: 同Sector Hw Address

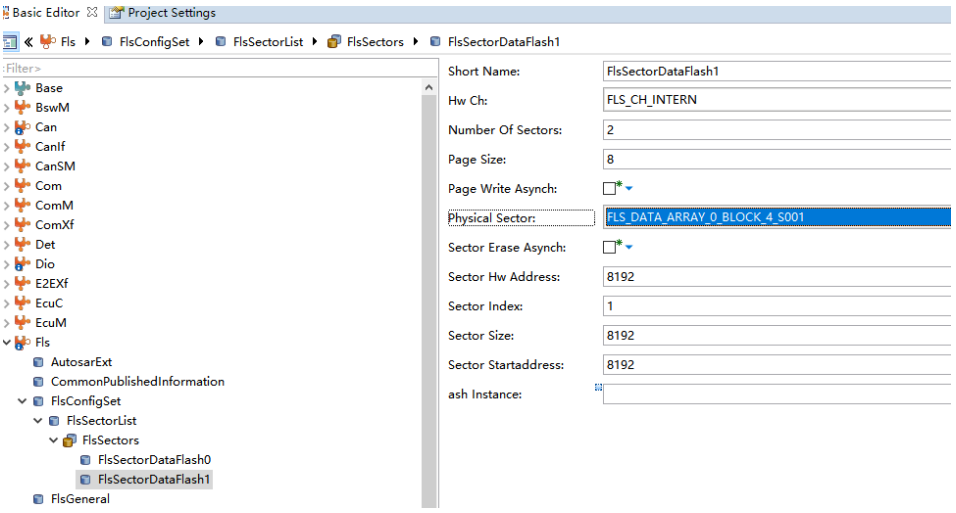
配置第一块

从DFlash的S000开始，就用一块，number=1

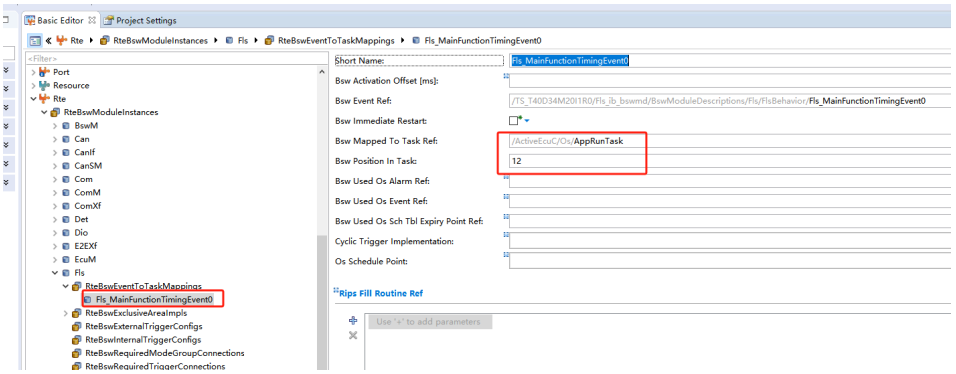


配置第二块

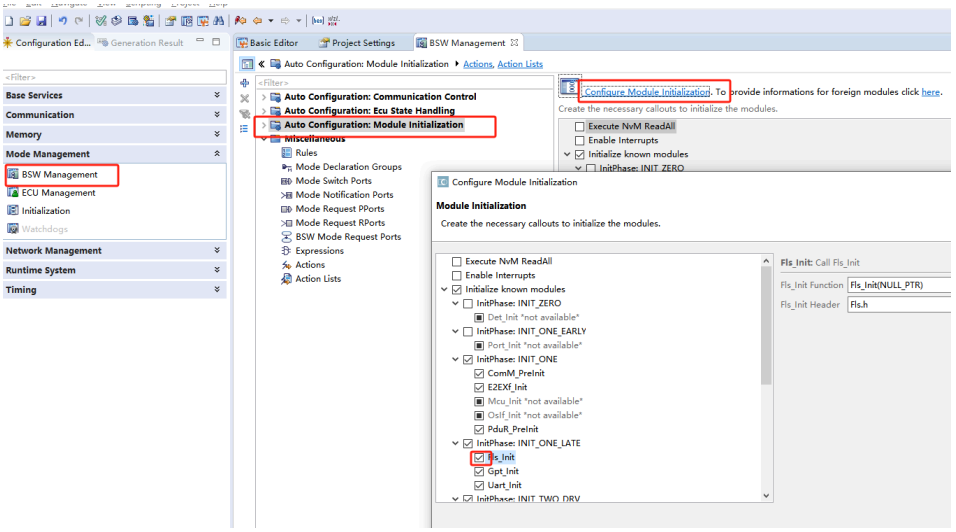
从DFlash的S001开始，就用两块，number=2 ，（所以就是用了S001和S002）



三、添加Rte的mapping



四、BswM添加Fls_Init

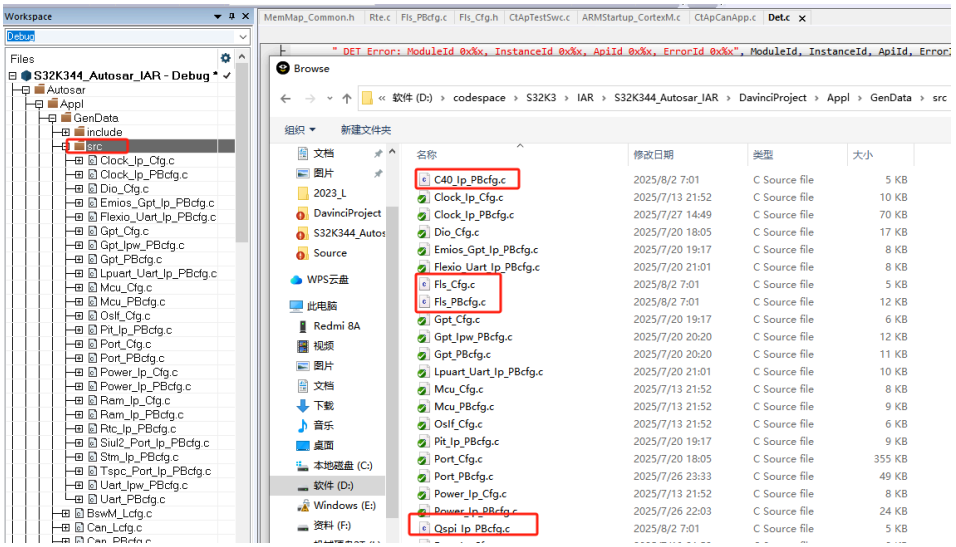


五、生成模块代码

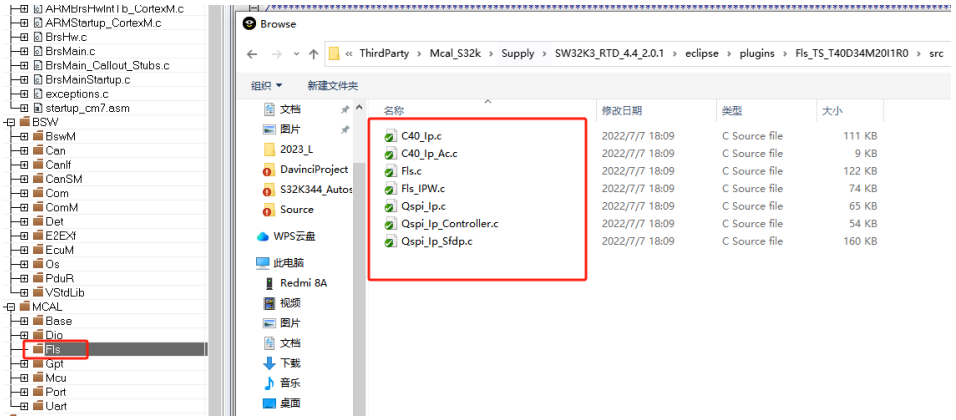
生成Fls\Rte\BswM\MemMap

六、IAR添加Fls相关模块文件

6.1、添加davinci生成的Fls GenData文件

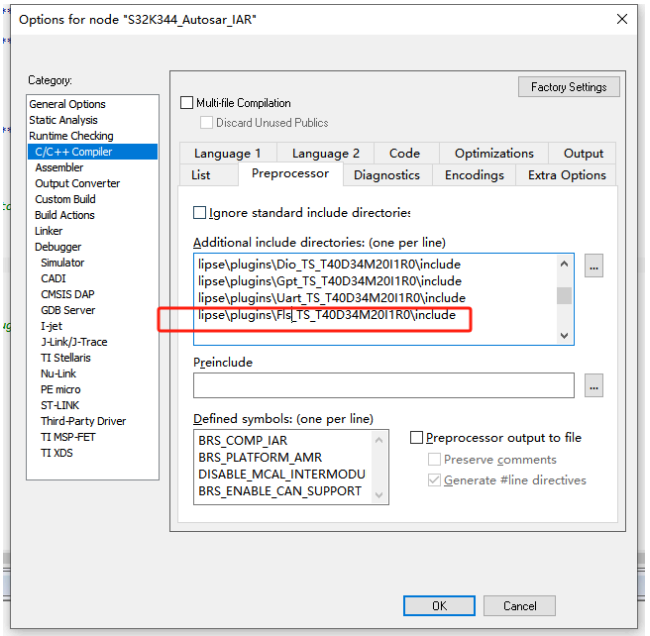


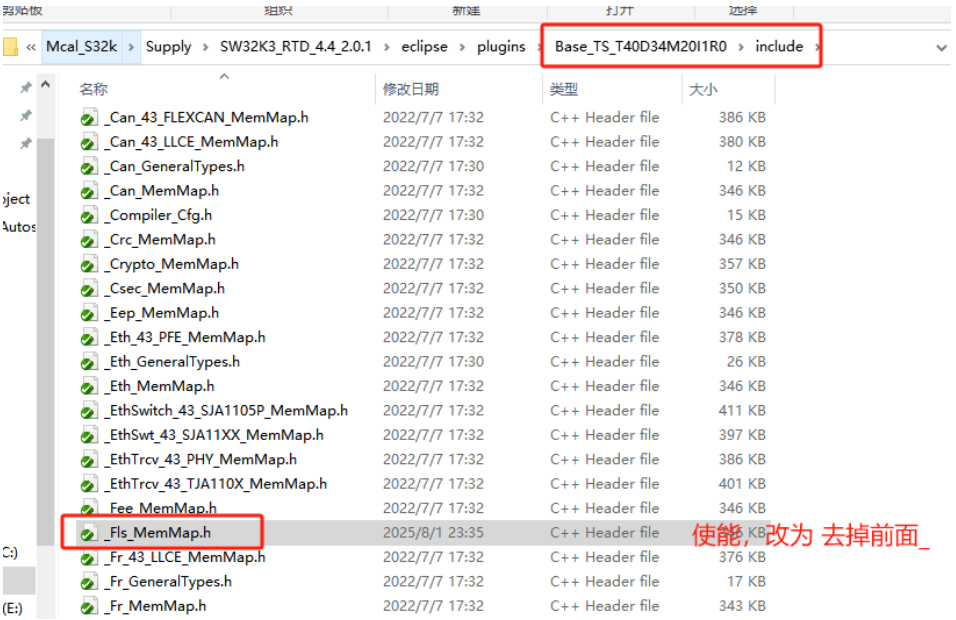
6.2、添加MCAL的Fls



添加头文件路径到IAR

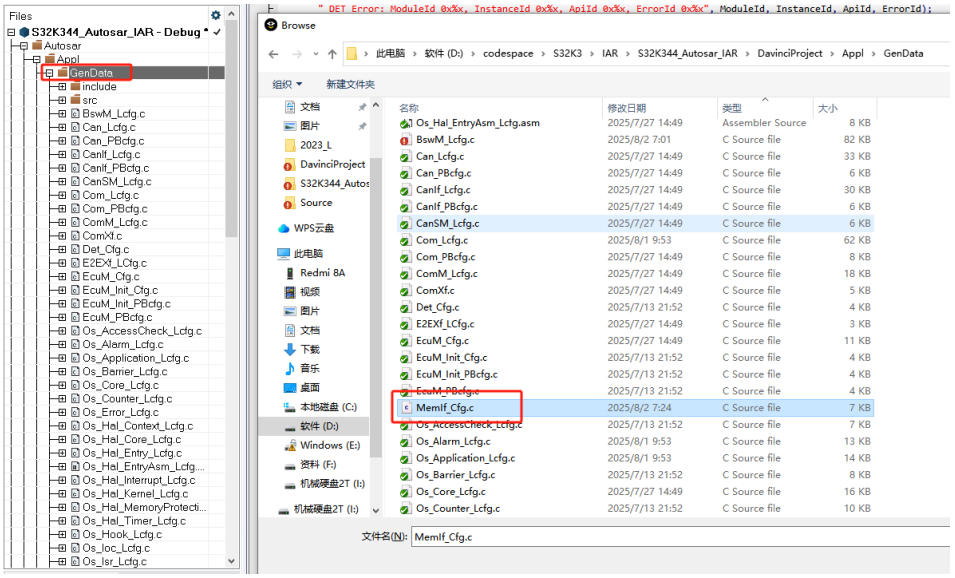
$PROJ_DIR$\CBD2300384_D00\ThirdParty\Mcal_S32k\Supply\SW32K3_RTD_4.4_2.0.1\eclipse\plugins\Fls_TS_T40D34M



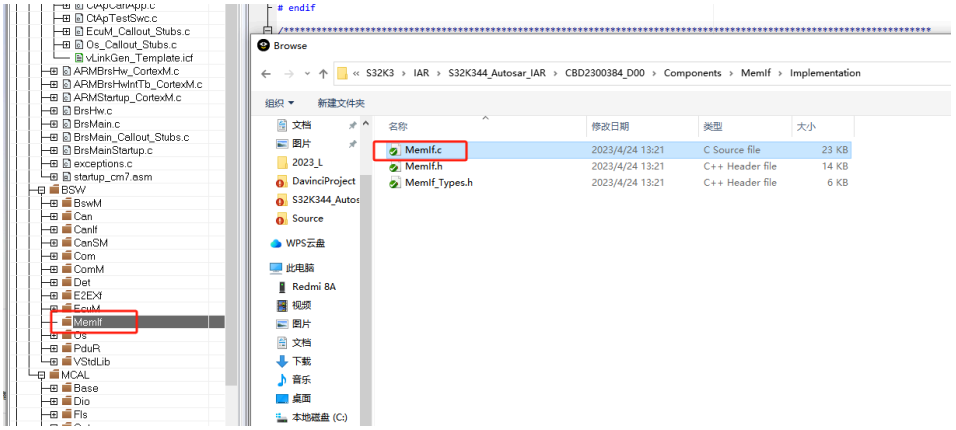


6.3、暂不添加davinci生成的MemIf GenData文件

暂不添加，下一节再添加，否则编译报错



6.4、添加BSW的MemIf

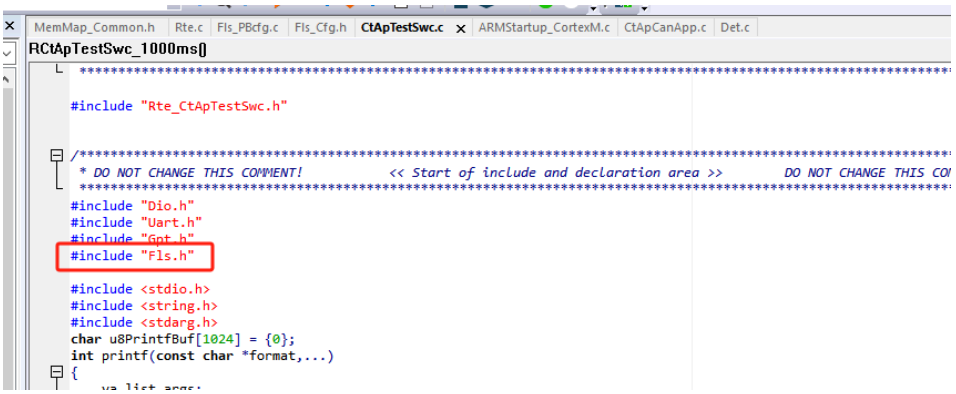


同6.2方式添加头文件路径到IAR

$PROJ_DIR$\CBD2300384_D00\Components\MemIf\Implementation

七、添加测试代码

CtApTestSwc.c 里面

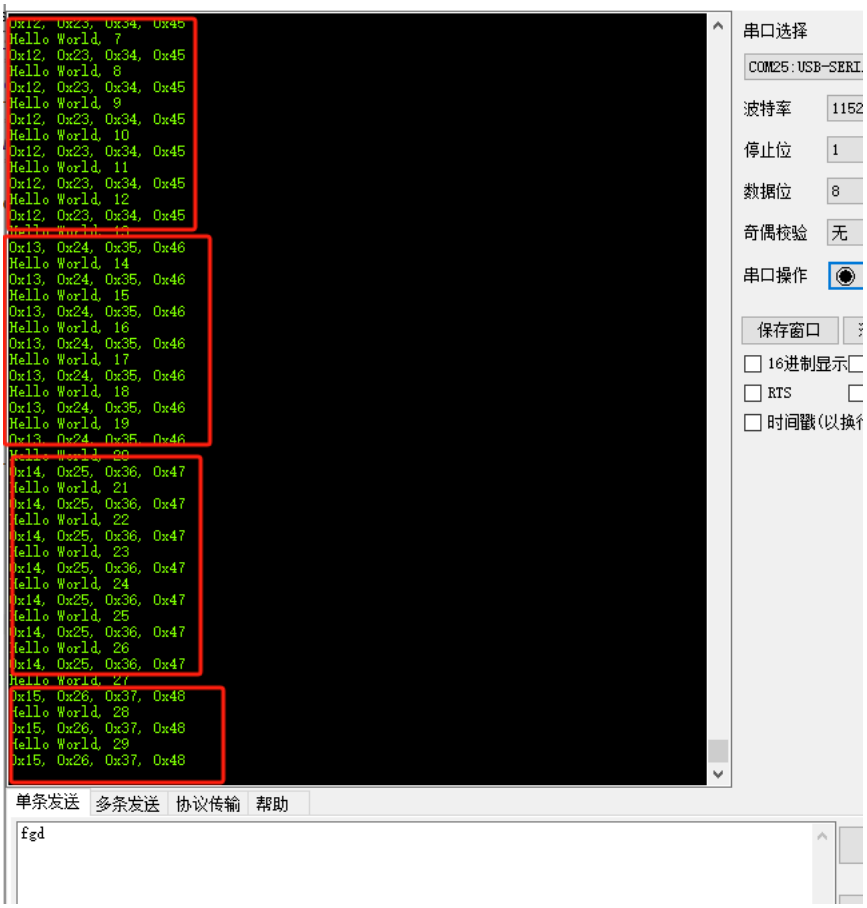


```c
#include "string.h"
uint8 au8data[8] = {0x11,0x22,0x33,0x44};
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << End of documentation area >>                    DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************************/

FUNC(void, CtApTestSwc_CODE) RCtApTestSwc_1000ms(void) /* PRQA S 0624, 3206 */ /* MD_Rte_0624, MD_Rte_3206 */
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           << Start of runnable implementation >>             DO NOT CHANGE THIS COMMENT!
 * Symbol: RCtApTestSwc_1000ms
 **********************************************************************************************************************/
  Dio_FlipChannel(DioConf_DioChannel_DioChannelPTA31_LED3_BLUE);
  Dio_FlipChannel(DioConf_DioChannel_DioChannelPTB18_LED4_YELLOW);
  //Uart_SyncSend(0, "Hello World\r\n",strlen("Hello World\r\n"),0xFFFF);
  static unsigned short u16Cnt = 0;
  printf("Hello World, %d\r\n",u16Cnt++);

  static unsigned char u8FlsFlag = 0;
  if(u8FlsFlag++ > 5){
    u8FlsFlag = 0;
    Std_ReturnType Ret = Fls_Erase(0x4000,8192);
    if (0 == Ret)
    {
      while (Fls_GetStatus() != MEMIF_IDLE)
      {
        Fls_MainFunction();
      }
    }
    au8data[0]++;
    au8data[1]++;
    au8data[2]++;
    au8data[3]++;
    Ret = Fls_Write(0x4000, au8data, 8);
    if (0 == Ret)
    {
      while (Fls_GetStatus() != MEMIF_IDLE)
      {
        Fls_MainFunction();
      }
    }
  }
  uint8 u8Temp[4] = {0x00,0x00,0x00,0x00};
  memcpy(u8Temp, (void*)0x10004000, 4);
  printf("0x%02x, 0x%02x, 0x%02x, 0x%02x\r\n", u8Temp[0],u8Temp[1],u8Temp[2],u8Temp[3]);
```

擦除 0x4000, 就是此操作第二块配置的dataflash

写数据自增

写数据4字节

通过memcpy绝对地址打印该四字节

测试结果符合预期，每5次数据自增完成